

Den Glade Skorpe

Et moderne pizzeria med online bestilling

Fagprøve – Webudvikleruddannelsen
Medieskolerne / Media College Denmark

Vigtige generelle oplysninger om eksamen

1. Prøvens karakter og selvstændighed

Den afsluttende projektopgave er en del af uddannelsens prøve og skal udarbejdes individuelt. Besvarelsen skal være elevens egen og selvstændige løsning i overensstemmelse med gældende regler for erhvervsuddannelser.

Projektet danner grundlag for den efterfølgende mundtlige prøve, hvor eleven skal kunne redegøre for valg, løsninger og anvendt kode. Institutionen forbeholder sig ret til at stille uddybende spørgsmål ved den mundtlige prøve for at sikre, at projektet er elevens eget arbejde.

2. Afvikling af projektperioden

Projektet løses inden for den fastsatte projektperiode. Arbejdet kan udføres på skolen eller hjemme efter eget valg. Vælger du at arbejde hjemme, sker dette på eget ansvar. Skolen kan ikke give forlænget tid ved tekniske problemer på eget udstyr eller netværk.

Du skal selv sikre:

- at dit udstyr fungerer
- at der løbende tages backup
- at projektet kan afleveres rettidigt

Manglende backup eller tekniske problemer på eget udstyr giver ikke ret til forlænget tid.

3. Hjælpemidler

Alle hjælpemidler, der normalt anvendes i undervisningen, er tilladt, medmindre andet er angivet. Det er tilladt at anvende tredjepartspakker, biblioteker og frameworks, forudsat at:

- de tydeligt kildehenvises i dokumentationen
- du selvstændigt kan redegøre for deres anvendelse
- de ikke udgør hovedparten af løsningen

Indhold fra tredjepart indgår ikke som selvstændig bedømmelse af dine kompetencer. Eventuel brug af AI-baseret kodeassistance skal angives (se rapportafsnittet).

Opgavebeskrivelse

Denne fagprøve er udformet, så du kan demonstrere dine tekniske og faglige kompetencer i overensstemmelse med målene for webudvikleruddannelsen.

Den Glade Skorpe er et pizzeria, der er kendt for saftige pizzaer med sprøde skorper. De har fået lavet et design til deres hjemmeside og ønsker den nu udviklet, så de kan tiltrække flere kunder.

Du skal udvikle en moderne og funktionel webapplikation, hvor gæster kan se menuen, åbne den enkelte ret, vælge størrelse og lægge retter i en kurv. Applikationen skal samtidig fungere som et backoffice for personalet, hvor de bl.a. kan tilføje, opdatere og slette medarbejdere. Løsningen skal på sigt kunne skaleres, men mobiludgaven er den vigtigste, da langt de fleste kunder forventes at bestille fra mobilen.

Formålet med løsningen er at:

- præsentere pizzeriaet og hele menukortet for potentielle gæster
- give kunder mulighed for at vælge retter, tilpasse dem og samle dem i en kurv (online bestilling)
- give personalet adgang til et backoffice, hvor data (fx medarbejdere) kan administreres

Løsningen skal demonstrere dine kompetencer inden for frontend-udvikling, integration af API, datahåndtering, brugeroplevelse og administrationsfunktionalitet.

Kravspecifikation og generelle krav

- Med opgaven er der udleveret skrifttyper, billedmateriale og en designfil (Figma), som viser layout samt eventuelle effekter. Designet skal som udgangspunkt følges i opsætning og visuel struktur.
- Kunden accepterer ændringer i designet, hvis du kan argumentere fagligt for, at ændringen forbedrer brugeroplevelse, funktionalitet eller tilgængelighed.
- Elementer, der fremgår af designet men ikke eksplicit er beskrevet i opgaven, skal som udgangspunkt implementeres. Omvendt kan der være funktionelle krav, som ikke er visualiseret – her udarbejder du selv en løsning i den visuelle stil.
- Løsningen skal udvikles med HTML, CSS og JavaScript og skal benytte et moderne frontend-framework (fx React, Next.js eller tilsvarende). Du må gerne anvende Sass/SCSS eller tilsvarende.
- Koden skal være semantisk korrekt og struktureret med fokus på:
 - genanvendelige komponenter
 - overskuelig mappestruktur
 - læsbar og vedligeholdelsesvenlig kode
- Løsningen skal overholde gængse standarder for tilgængelighed (accessibility), responsivitet og grundlæggende SEO-principper.
- Løsningen skal udvikles **mobile-first** og fungere optimalt fra ca. 390px og opefter. Mobiludgaven er det primære krav; desktop/backoffice vises på større skærm.
- Der skal implementeres relevant fejlhåndtering, herunder:
 - 404-side ved ugyldig route
 - håndtering af fejl ved API-kald
 - brugervenlige fejlbeskeder ved ugyldigt input
- Alle formularer skal indeholde relevant validering (både teknisk og brugervenlig feedback).
- Løsningen skal have tydeligt fokus på brugeroplevelse (UX): informationsarkitektur og navigation, intuitiv interaktion, tydelig feedback ved brugerhandlinger, kontrast/læsarhed/visuelt hierarki samt ydeevne og indlæsningstid.

Obligatoriske opgaver

Følgende skal være løst, for at opgaven kan bestås:

1. Mobile-first frontend i forhold til Figma.
2. Forside med alle retter samt en filtreringsfunktion (filtrér efter kategori).
3. Detalside for den enkelte ret (dish): valg af evt. størrelse og "Tilføj til kurv" (gemmes i localStorage).
4. Personalet: visning af restaurantens medarbejdere.
5. Kontakt: formular med validering, der sender navn, emne og besked til serveren.
6. Kurv: vis de bestillinger, du har lagt i kurven via localStorage.
7. Backoffice (ikke et mobilkrav): Medarbejdere (Employees) – opret (POST), redigér (PUT) og slet (DELETE).

API / Server

Løsningen skal integrere det udleverede API (backend).

Serveren kører lokalt på `http://localhost:3042`. Følg den medfølgende readme for installation og opstart, og test endpoints (fx i Postman/browser) før du integrerer dem i frontend.

API'et håndterer data for bl.a.:

- Retter (dishes)
- Medarbejdere (employees)
- Beskeder fra kontaktformularen (messages)
- Ordre (orders) – ved tilvalg

Krav til API-integration

Du skal demonstrere korrekt anvendelse af REST-principper:

- GET → hente data (fx retter og medarbejdere)
- POST → oprette data (fx ny medarbejder, kontaktbesked, ordre)
- PUT/PATCH → opdatere data (fx redigér medarbejder)
- DELETE → slette data (fx slet medarbejder)

Integration skal ske via asynkrone kald (fetch/axios eller tilsvarende). Der skal implementeres:

- Indlæsningstilstand (loading state)
- Fejlhåndtering ved mislykkede requests
- Brugervenlig feedback ved succes/fejl

Autentifikation (tilvalg)

Backoffice kan beskyttes via login. Authentication er et tilvalg: du kan slå auth til på serveren og implementere en Sign In, så kun loggede brugere har adgang til backoffice. Ved login skal token/session håndteres korrekt.

Ændringer i API'et

Du må udvide eller ændre det eksisterende API, hvis det er relevant for opgavens krav. Ændringer kræver forhåndsgodkendelse fra underviser og skal dokumenteres i rapporten med: hvad der er ændret/tilføjet, hvorfor det var nødvendigt, og hvordan det påvirker frontend-integrationen.

Websitets indhold (sider)

Applikationen bygges op om følgende routes. Diagrammet i Figma illustrerer hensigten – det skal ikke følges slavisk.

Obligatoriske routes

- / – Forside
- /dish/:id – Detaljeside for en ret
- /employees – Personalet
- /contact – Kontakt
- /basket – Kurv
- /backoffice – Backoffice (forside)
- /backoffice/employees – Administrér medarbejdere

Tilvalgs-routes

- /backoffice/dishes – Administrér retter
- /backoffice/messages – Beskeder fra kontaktformularen

- /backoffice/orders – Ordre

Indhold på alle sider (frontend)

Header

- Mørk header med logo ("Den Glade Skørpe")
- Kurv-ikon med antal varer (badge)
- Burgermenu med navigation til undersiderne

Footer

- Logo
- Kontaktinformation (email, telefon, adresse)
- [evt. åbningstider / SoMe-links – tjek Figma]

Forside – /

- Header (logo, kurv, burgermenu) og en kort velkomst/introtekst om Den Glade Skørpe
- "Vælg kategori": filtrering af menuen (fx Pizzaer, Indbagte pizzaer, Durum ruller)
- Visning af alle retter (hentes fra API'et), hver med billede og navn
- Filtreringsfunktionen skal opdatere listen dynamisk uden reload
- Klik på en ret fører til dens detaljeside (/dish/:id)
- Footer med kontaktinfo

Detaljeside – /dish/:id

- Vis den valgte ret: billede, navn, beskrivelse og pris (hentes fra API'et ud fra id)
- Valg af evt. størrelse
- "Tilføj til kurv" – varen gemmes i localStorage
- Tydelig feedback når en ret er lagt i kurven
- Tilvalg: mulighed for at tilføje ekstra ingredienser på detaljesiden (se *Tilvalgsopgaver*)

Personalet – /employees

- Visning af restaurantens medarbejdere (hentes fra API'et)
- Hver medarbejder vises fx med billede, navn og titel/rolle
- Loading- og error states ved hentning

Kontakt – /contact

- Kontaktformular med felterne navn, emne og besked
- Validering af felterne (teknisk + brugervenlig feedback)
- Ved gyldig indsendelse sendes data til serveren (POST)
- Ved succes vises en kvitteringsside/-besked ("*Tak for din besked!*")

Kurv – /basket

- Vis de retter, brugeren har lagt i kurven (læses fra localStorage)
- Vis relevante detaljer pr. linje (fx navn, størrelse, evt. ekstra ingredienser, antal, pris) samt en samlet sum
- Tom kurv håndteres pænt ("Din kurv er tom")
- Tilvalg: afgiv ordre ved at sende kurvens indhold til serveren (POST). Ved succes vises kvittering ("*Tak for din bestilling*")

Backoffice – /backoffice og /backoffice/employees

Backoffice behøver kun fungere i desktop-/større skærmvisning. Administrator skal kunne administrere medarbejdere:

- Se oversigt over alle medarbejdere
- Oprette ny medarbejder (POST)
- Redigere eksisterende medarbejder (PUT/PATCH)
- Slette medarbejder (DELETE)
- Feedback og fejlhåndtering ved hver handling

Tilvalgsopgaver

Du skal vælge minimum 1 af følgende tilvalg. Du må gerne vælge flere, hvis de obligatoriske krav er opfyldt og stabile, og du har styr på prioriteringen:

1. Responsivt breakpoint > 1024px (desktop-layout for frontend).
2. Tilføjelse af ekstra ingredienser på detaljesiden. Eksempel findes i Figma; her må du gerne ændre i designet for at vise din egen løsning.
3. Backoffice – Beskeder (/backoffice/messages): vis beskeder sendt via kontaktformularen, med read/unread-status.
4. Backoffice – Retter (/backoffice/dishes): opret (POST), redigér (PUT) og slet (DELETE) retter.
5. Afgiv ordrer via serveren: send (POST) bestillingerne fra kurven til serveren.
6. Authentication: slå auth til på serveren og implementér Sign In, så backoffice beskyttes.

Design

Figma-design: <https://www.figma.com/design/yzjuDfwFzngz8EySrOXsf6/Den-Glade-Skorpe?node-id=0-1&m=dev>

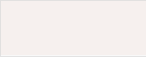
Designet indeholder mobil-frames (Forside, Detaljeside, Burgermenu, Personalet, Kontakt, Kurv, kvitteringssider samt eksempler på “valgt kategori” og “tilføj ingredienser”) og fire backoffice-frames (medarbejdere, beskeder, ordrer og retter).

Skrifttyper

1. **Overskrifter / knapper:** Just Another Hand – fontsource.org/fonts/just-another-hand
2. **Brødtekst:** Kurale – fontsource.org/fonts/kurale

Farver

Aflæst direkte fra designet (der er ikke defineret farve-variabler i filen – verificér gerne mod Figma):

	#4A4A4A	Mørk grå/koksgrå – header, footer, knapper, primær kontrast
	#F6F0EE	Creme – sektions- og kort-baggrunde
	#FFFFFF	Hvid – sidebaggrund og tekst på mørk flade
	#000000	Sort – brødtekst

Design-noter

- Designet er mobile-first; backoffice er tænkt til større skærm.
- Navigationsdiagrammet i Figma viser hensigten og skal ikke følges slavisk.

Billeder/grafik

Du har fået udleveret grafik (Figma) og fotos. Visse billeder (som knytter sig til data, fx retter og medarbejdere) hentes fra API'et. Hvor der mangler billeder, må du selv finde passende "free" billeder på fx Unsplash, Pexels eller Pixabay. Tilpas selv billedstørrelser til layoutet hvor nødvendigt.

Dokumentation (rapport)

Du skal udarbejde en rapport sideløbende med opgaveløsningen. Rapporten indgår i eksaminationsgrundlaget og danner udgangspunkt for den mundtlige prøve. Den afleveres som .md- eller .pdf-fil i projektets rodmappe.

Tidsplan, estimat og todo-liste

Lav projektplanlægningen som noget af det første. Nedbryd projektet i mindre opgaver (tasks/issues) og estimér hver opgave. Følg løbende op på planlagt tid, faktisk anvendt tid og justeringer. Daglig planlægning og eksekvering (fx i Trello, GitHub Projects eller lignende agilt værktøj) ses gerne – evt. via link i rapporten.

Rapportens indhold

1. Vurdering af egen indsats

Hvad gik godt? Hvad var udfordrende – og hvordan løste du det? Hvad ville du gøre anderledes? Hvordan har du udviklet dig fagligt? Formålet er at vise refleksion og faglig forståelse.

2. Tidsplan og proces

- Oversigt over planlagte og udførte opgaver
- Estimeret vs. faktisk tid
- Dokumentation (fx Trello, GitHub Projects, Gantt-chart, foto af tavle) – kan vedlægges som bilag

3. Tech stack

Beskriv og begrund kort dine valg: frontend-framework, CSS-metode (Sass, Tailwind, styled components mv.) samt biblioteker/npm-pakker.

4. Faglige valg og dokumentation

- Arkitekturvalg og strukturering af komponenter
- State management (inkl. håndtering af kurv i localStorage)
- API-håndtering
- Eventuelle ændringer i backend (hvad og hvorfor – kræver godkendelse; vedlæg link hvis hostet)

5. Tilvalgsopgaver

Angiv tydeligt, hvilke tilvalg du har løst.

6. Anvendelse af tredjepart og AI

Angiv alle tredjepartspakker/biblioteker, færdige scripts/plugins samt eventuel brug af AI-baseret kodeassistance. Ved brug af AI skal du kort beskrive hvordan det er anvendt, dokumentere at du selv har forstået og tilpasset koden, og kunne redegøre for den mundtligt. Manglende angivelse kan betragtes som plagiat.

7. Testoplysninger

- Evt. URL til løsningen

- Brugernavne og passwords (fx til backoffice)
- Eventuelt GitHub-link (privat indtil aflevering er afsluttet)

8. Særlige punkter til bedømmelse

Beskriv eventuelle særlige løsninger eller prioriteringer, du ønsker fremhævet.

Rapportens forside

På forsiden skal fremgå: opgavens navn, dit navn og holdnummer, skole samt afleveringsdato – og nødvendige brugernavne/adgangskoder. Samt følgende erklæring:

Jeg bekræfter hermed, at jeg selvstændigt og uden uretmæssig hjælp har udviklet det afleverede eksamensprojekt i overensstemmelse med gældende regler for prøven.

Erklæringen underskrives (fx digital signatur eller indsat billede) og dateres.

Bilag

Du kan vedlægge bilag, fx uddrag af særlige kode-eksempler, din tidsplan samt fotos/screenprints, der dokumenterer arbejdsprocessen.

Prioritering

Prioritér opgaverne efter denne rækkefølge (især punkt 1–4 er afgørende for at bestå):

1. Projektstruktur og opsætning: Git-repo, routing, komponentstruktur, styling-setup og grundlæggende sider efter Figma (mobile-first).
2. Grundlæggende layout og design: header/burgermenu/footer på alle sider; forside + menu matcher Figma mest muligt.
3. API-integration (GET): hent og vis retter på forsiden + detaljeside, og medarbejdere på /employees. Loading- og error states.
4. Kurv-flow: detaljeside med størrelsesvalg → tilføj til kurv (localStorage) → vis kurv. Tydelig feedback.
5. Kontaktformular (POST): validering og afsendelse til server med succes-/fejlfeedback.
6. Backoffice – Medarbejdere (CRUD): oversigt + opret/redigér/slet med feedback og fejlhåndtering.
7. Fejlhåndtering og UX-finish: 404-side, tomme lister (fx "tom kurv"), bedre microcopy og brugerfeedback.
8. Tilvalgsopgaver: først når alt ovenstående er stabilt og afleveringsklart.
9. Ekstra forbedringer: performance, små animationer, ekstra filtre, dashboards – kun hvis kernekravene er 100% på plads.

HUSK også rapporten med dokumentation af, hvordan du har struktureret, planlagt og estimeret projektet.

Aflevering

Udlevering: [dato + kl.] **Aflevering:** [dato + kl.]

Din aflevering skal indeholde:

1. Din kildekode (js, css, html, package.json osv.) – alt hvad du benytter til at lave projektet.
2. En readme.md, der beskriver, hvad der skal til for at starte projektet.
3. En rapport (.pdf eller .md).

Afleveringsformat (aftal med din underviser, og forbered gerne begge):

- Det hele som én samlet zip-fil UDEN mappen node_modules, eller
- Det hele som et Git-repository (UDEN node_modules)
- **VIGTIGT:** Redigér ikke i din løsning efter afleveringsfristen er udløbet. Vil du arbejde videre efter aflevering, så lav en kopi til formålet.

Backend

Backend/API skal som udgangspunkt ikke afleveres, medmindre du (efter aftale) har ændret i det udleverede API. Husk i så fald at argumentere for ændringerne i rapporten.

Din aflevering skal godkendes

Når du har afleveret, skal du informere din underviser, som tjekker, at alt er afleveret og kan downloades/unzippes. Afvent godkendelsen, inden du holder fri.

Mundtlig eksamination

Til eksaminationen præsenterer du din løsning og skal kunne redegøre for enkeltelementer – fx kode, valg af metodik, navigationselementer eller datalayout. Øvrige emner inden for uddannelsens fagområde kan indgå. Mød i god tid på eksamensdagen, klargør maskine og browser, og test på forhånd, at din opsætning virker.

God arbejdslyst!